

# Algorithms Cheat Sheet

Last Update: June 28, 2026

## Contents

|                                                                     |           |                                                                |           |
|---------------------------------------------------------------------|-----------|----------------------------------------------------------------|-----------|
| <b>1 Common Pitfalls &amp; General Tips</b>                         | <b>3</b>  | <b>10 Combinatorics and Coin Change</b>                        | <b>15</b> |
| 1.1 Exam Traps                                                      | 3         | 10.1 Dice Combinations [CSES]                                  | 15        |
| <b>2 Basics &amp; Essential Snippets</b>                            | <b>3</b>  | 10.2 Minimizing Coins [CSES]                                   | 15        |
| 2.1 Template Setup & I/O Optimizations                              | 3         | 10.3 Coin Combinations I & II [CSES]                           | 16        |
| 2.2 Basic Structures, Vectors, and Matrices                         | 3         | 10.4 Removing Digits [CSES]                                    | 16        |
| 2.3 Backtracking to Find DP Solutions                               | 3         | 10.5 Problem B: Restaurant Orders [Kattis]                     | 16        |
| <b>3 Backtracking, Pruning &amp; Branch and Bound</b>               | <b>4</b>  | <b>11 Knapsack Variants</b>                                    | <b>17</b> |
| 3.1 Core Concept: The Backtracking Skeleton                         | 4         | 11.1 0/1 Knapsack Problem                                      | 17        |
| 3.2 Branch and Bound & Pruning: Traveling Salesperson Problem (TSP) | 4         | 11.2 Book Shop [CSES]                                          | 17        |
| 3.3 N-Queens Problem                                                | 5         | 11.3 Money Sums & Two Sets II [CSES]                           | 18        |
| <b>4 Greedy Fundamentals &amp; Sorting</b>                          | <b>6</b>  | 11.4 Problem A: Knapsack [Kattis]                              | 18        |
| 4.1 Greedy Heuristics: The Knapsack Problem                         | 6         | 11.5 Problem C: Presidential Elections [Kattis]                | 19        |
| 4.2 Shopaholic [Kattis]                                             | 7         | <b>12 Jumps and Sequences 1D</b>                               | <b>20</b> |
| 4.3 Frosh Week (Inversion Counting) [Kattis]                        | 7         | 12.1 Maximum Weight Independent Set (MWIS) on a Path           | 20        |
| <b>5 Interval Scheduling &amp; Placement</b>                        | <b>7</b>  | 12.2 Frog 1: Jump 1 or 2 Stones                                | 20        |
| 5.1 Greedy Scheduling (Activity Selection)                          | 7         | 12.3 Frog 2: Jump up to K Stones                               | 20        |
| 5.2 Assigning Workstations [Kattis]                                 | 8         | 12.4 Problem D: Spiderman's Workout [Kattis]                   | 21        |
| 5.3 Birds on a Wire [Kattis]                                        | 9         | <b>13 Grid, Matrices and Shapes</b>                            | <b>21</b> |
| <b>6 Priority Queues &amp; Data Structures</b>                      | <b>9</b>  | 13.1 Matrix Chain Multiplication (Interval DP)                 | 21        |
| 6.1 Huffman Codes                                                   | 9         | 13.2 Grid Paths [CSES]                                         | 22        |
| 6.2 Boiling Vegetables [Kattis]                                     | 10        | 13.3 Rectangle Cutting [CSES]                                  | 22        |
| <b>7 Minimum Spanning Trees (MST)</b>                               | <b>10</b> | 13.4 Minimum Sum in a Triangle                                 | 22        |
| 7.1 Prim's Algorithm (Adjacency Matrix)                             | 10        | 13.5 Maximal Square (All 1s)                                   | 23        |
| 7.2 Prim's Algorithm (Priority Queue and Adjacency List)            | 11        | <b>14 Subsequences and Strings</b>                             | <b>23</b> |
| 7.3 Kruskal's Algorithm                                             | 12        | 14.1 Increasing Subsequence [CSES]                             | 23        |
| 7.4 Lost Map [Kattis]                                               | 13        | 14.2 Problem E: Increasing Subsequence [Kattis]                | 23        |
| <b>8 Ad-Hoc / Mathematical Greedy</b>                               | <b>13</b> | 14.3 Longest Common Subsequence (LCS)                          | 24        |
| 8.1 Ants [Kattis]                                                   | 13        | 14.4 Word Break Problem                                        | 24        |
| <b>9 Strategies and Base Concepts</b>                               | <b>14</b> | <b>15 Graphs and Short Path [DP]</b>                           | <b>24</b> |
| 9.1 Dynamic Programming: General Strategy                           | 14        | 15.1 Bellman-Ford Algorithm                                    | 24        |
| 9.2 Sequence Alignment (Edit Distance)                              | 14        | 15.2 Problem F & G: Bellman-Ford with Negative Cycles [Kattis] | 25        |
| 9.3 Optimal Binary Search Tree (OBST)                               | 14        | 15.3 Floyd-Warshall (All-Pairs Shortest Path)                  | 26        |
|                                                                     |           | <b>16 Mathematics and State Machines</b>                       | <b>26</b> |
|                                                                     |           | 16.1 Array Description [CSES]                                  | 26        |
|                                                                     |           | 16.2 Removal Game [CSES]                                       | 26        |
|                                                                     |           | 16.3 Catalan Numbers (BSTs, Parentheses, Triangulations)       | 27        |

|      |                                                          |           |
|------|----------------------------------------------------------|-----------|
| 16.4 | Count Derangements . . . . .                             | 27        |
| 16.5 | Binomial Coefficient (Pascal's Triangle 1D) . . . . .    | 27        |
| 16.6 | Decode Ways . . . . .                                    | 27        |
| 17   | <b>Advanced Topics (Bitmask and Intervals) . . . . .</b> | <b>28</b> |
| 17.1 | Projects (Weighted Interval Scheduling) [CSES] . . . . . | 28        |
| 17.2 | Elevator Rides (Bitmask DP) [CSES] . . . . .             | 28        |

## 1. Common Pitfalls & General Tips

### 1.1. Exam Traps

Save yourself hours of debugging during the exam by watching out for these frequent mistakes:

- **The Memory Trap (MLE):** Using `#define int long long` globally is a common trick to blindly prevent integer overflow. However, a 64-bit integer doubles your memory usage compared to a 32-bit integer. On strict problems (like CSES *Book Shop*), this will instantly cause a Memory Limit Exceeded (MLE) error. Use standard 32-bit `int` for massive DP tables.
- **The Indexing Offset Trap:** Pay close attention to how items are indexed. On many graph problems or 2D DP matrices (like 0/1 Knapsack or Longest Common Subsequence) use **1-based indexing** for the state variables (e.g., `dp[i][w]` represents the first  $i$  items). This allows row 0 to safely act as the base case. However, your input arrays (like `items[i-1]`) use standard **0-based indexing**. Mixing these up is the number one cause of Out Of Bounds errors.
- **The "Infinity" Overflow:** When initializing DP tables for minimization problems, you often need an "Infinity" value. Do **not** use `INT_MAX` if your state transition involves adding to it (e.g., `dp[i] = dp[prev] + cost`). `INT_MAX + 1` will overflow into a massive negative number, completely breaking your `min()` logic. Instead, use a "safe infinity" like `1e9` for 32-bit integers, or `1e15` for `long long`.

## 2. Basics & Essential Snippets

### 2.1. Template Setup & I/O Optimizations

**Note:** the bits library is a GCC-specific header that includes almost every standard library in C++. It slows down compilation slightly but saves time typing during an exam.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Replaces all 'int' with 'long long' to prevent integer overflow globally.
5 // WARNING: If you use this, your main function MUST be declared as int32_t.
6
7 #define int long long
8
9 signed main() {
10     // I/O Optimizations: Disconnects C and C++ standard streams.
11     // Makes cin/cout almost as fast as scanf/printf.
12     ios_base::sync_with_stdio(false);
13     cin.tie(NULL);
14
15     int num_elements, target_value;
16
17     // Reading until End of File (EOF).
18     // Useful when the problem doesn't specify the number of test cases.
19     while (cin >> num_elements >> target_value) {
20         // Process each test case here
21     }
22

```

```

23     return 0;
24 }

```

### 2.2. Basic Structures, Vectors, and Matrices

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 // 1. Defining a Struct
8 struct Player {
9     int score;
10    int penalties;
11 };
12
13 // 2. Comparing Two Elements (Custom Sorting)
14 // Returns true if 'a' should be placed BEFORE 'b' in the sorted array
15 bool comparePlayers(const Player& a, const Player& b) {
16     if (a.score != b.score) {
17         return a.score > b.score; // Primary rule: Sort descending by score
18     }
19     return a.penalties < b.penalties; // Tie-breaker: Sort ascending by penalties
20 }
21
22 void standardDataStructures() {
23     int num_items = 10;
24     int rows = 5, cols = 5;
25
26     // 3. Defining a Vector (1D Array)
27     // Space Complexity: O(N)
28     // Creates a vector of size 10, with all elements initialized to 0
29     vector<int> one_dimensional_array(num_items, 0);
30
31     // 4. Defining a Matrix (2D Vector)
32     // Space Complexity: O(Rows * Cols)
33     // Creates a 5x5 matrix, with all elements initialized to -1
34     vector<vector<int>> matrix(rows, vector<int>(cols, -1));
35
36     // 5. Selecting Maximum Element
37     vector<int> numbers = {1, 5, 3, 9, 2};
38
39     // Time Complexity: O(N)
40     // Note: max_element returns an ITERATOR. We use '*' to get the actual value.
41     int max_value = *max_element(numbers.begin(), numbers.end());
42 }

```

### 2.3. Backtracking to Find DP Solutions

**Concept:** Once a DP table is filled, you start from the final optimal state (usually the bottom-right corner of the matrix) and trace backward. You compare the current state to

previous states to determine which “choice” led you to the current optimal value.

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 // Example: Recovering the items chosen in a 0/1 Knapsack Problem
7 // dp_table[i][w] stores the max value using the first 'i' items with capacity
  'w'.
8 vector<int> getChosenItems(const vector<vector<int>>& dp_table, const vector<
  int>& weights, int num_items, int max_capacity) {
9     // Space Complexity: O(N) to store the result
10    vector<int> chosen_items;
11
12    int current_weight = max_capacity;
13
14    // Time Complexity: O(N) - We backtrack step-by-step through the items
15    for (int i = num_items; i > 0; --i) {
16
17        // If the value in the DP table changed compared to the row above it,
18        // it means we MUST have included item 'i' to achieve this value.
19        if (dp_table[i][current_weight] != dp_table[i-1][current_weight]) {
20
21            // Note: DP table items are 1-indexed, but array weights are 0-
22            // indexed.
23            int original_item_index = i - 1;
24            chosen_items.push_back(original_item_index);
25
26            // Deduct the weight of the chosen item to move left in the DP
27            // table
28            current_weight -= weights[original_item_index];
29        }
30    }
31
32    // Optional: reverse(chosen_items.begin(), chosen_items.end());
33    // Do this if you need the items in the order they originally appeared.
34    return chosen_items;
35 }

```

### 3. Backtracking, Pruning & Branch and Bound

#### 3.1. Core Concept: The Backtracking Skeleton

Backtracking builds a solution step-by-step. We usually use a `std::vector` to represent our current path. We `push_back` a choice to move forward, recurse, and then `pop_back` to undo that choice and try the next one.

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 // A global or passed-by-reference vector to store all valid results
7 vector<vector<int>> all_valid_solutions;

```

```

8
9 // --- PRACTICAL BACKTRACKING SKELETON ---
10
11 void backtrack(int current_step, int target_length, vector<int>& current_path)
12 {
13     // 1. BASE CASE: Have we reached the required length or final state?
14     if (current_step == target_length) {
15         // Save a copy of the successful path
16         all_valid_solutions.push_back(current_path);
17         return;
18     }
19
20     // 2. ITERATE CHOICES: Loop through all possible options for the current
21     // step
22     int num_possible_choices = 10; // Example: numbers 0 through 9
23     for (int choice = 0; choice < num_possible_choices; ++choice) {
24
25         // 3. PRUNING: Is this choice legally allowed based on the problem
26         // rules?
27         // (e.g., "Has this number already been used?" or "Is this space safe
28         // ?")
29         bool is_valid = true; /* Replace with actual condition */
30
31         if (is_valid) {
32
33             // 4. DO: Add the choice to our path
34             current_path.push_back(choice);
35
36             // 5. RECURSE: Move to the next step
37             backtrack(current_step + 1, target_length, current_path);
38
39             // 6. UNDO (Backtrack): Remove the choice so the loop can try the
40             // next one
41             current_path.pop_back();
42         }
43     }
44 }
45
46 int main() {
47     vector<int> starting_path;
48     int target_solution_length = 5;
49
50     // Start at step 0 with an empty path
51     backtrack(0, target_solution_length, starting_path);
52
53     return 0;
54 }

```

#### 3.2. Branch and Bound & Pruning: Traveling Salesperson Problem (TSP)

**The Problem:** Find the shortest possible route that visits every city exactly once and returns to the origin city.

**Pruning:** We do not visit a city if it has already been visited.

**Branch & Bound:** If the cost of our current incomplete path is already greater than or

equal to the best complete path we've found so far, we stop searching this branch.

```

1 #include <iostream>
2 #include <vector>
3 #include <climits>
4
5 using namespace std;
6
7 // Time Complexity: Worst case O(N!) without Branch and Bound.
8 // With Branch and Bound, the time is drastically reduced but strictly remains
9 // O(N!) upper bound.
10 void solveTravelingSalesperson(
11     int current_city,
12     int cities_visited,
13     int current_cost,
14     vector<bool>& has_visited,
15     const vector<vector<int>>& distance_matrix,
16     int& best_possible_cost)
17 {
18     int total_cities = distance_matrix.size();
19
20     // --- BRANCH AND BOUND ---
21     // If our current path is already more expensive than a full path we
22     // found earlier, there is no point in continuing. Abandon this branch.
23     if (current_cost >= best_possible_cost) {
24         return;
25     }
26
27     // --- BASE CASE ---
28     // We have visited every city exactly once.
29     if (cities_visited == total_cities) {
30         // Add the distance to return to the starting city (city 0)
31         int cost_to_return = distance_matrix[current_city][0];
32         int final_path_cost = current_cost + cost_to_return;
33
34         // We already checked current_cost < best_possible_cost, but the
35         // return trip might push it over. Check one last time.
36         if (final_path_cost < best_possible_cost) {
37             best_possible_cost = final_path_cost;
38         }
39         return;
40     }
41
42     // --- RECURSIVE STEP ---
43     // Try visiting every other city from the current city
44     for (int next_city = 0; next_city < total_cities; ++next_city) {
45         // --- PRUNING ---
46         // Only proceed if we haven't visited this city yet, and a path
47         // actually exists
48         if (!has_visited[next_city] && distance_matrix[current_city][next_city]
49             > 0) {
50
51             // 1. DO (Make the choice)
52             has_visited[next_city] = true;

```

```

52         // 2. RECURSE (Move forward)
53         solveTravelingSalesperson(
54             next_city,
55             cities_visited + 1,
56             current_cost + distance_matrix[current_city][next_city],
57             has_visited,
58             distance_matrix,
59             best_possible_cost
60         );
61
62         // 3. UNDO (Backtrack to try the next loop iteration)
63         has_visited[next_city] = false;
64     }
65 }
66
67 int main() {
68     int num_cities;
69     cin >> num_cities;
70
71     // Space Complexity: O(N^2) for the distance matrix
72     vector<vector<int>> distance_matrix(num_cities, vector<int>(num_cities));
73     for (int i = 0; i < num_cities; ++i) {
74         for (int j = 0; j < num_cities; ++j) {
75             cin >> distance_matrix[i][j];
76         }
77     }
78
79     // Space Complexity: O(N) for visited tracking
80     vector<bool> has_visited(num_cities, false);
81     int best_possible_cost = INT_MAX;
82
83     // Start at city 0
84     has_visited[0] = true;
85
86     // Initial call: Start at city 0, 1 city visited, 0 initial cost
87     // Space Complexity: O(N) max recursion depth for the call stack
88     solveTravelingSalesperson(0, 1, 0, has_visited, distance_matrix,
89                             best_possible_cost);
90
91     cout << best_possible_cost << "\n";
92     return 0;
93 }

```

### 3.3. N-Queens Problem

**Concept:** Place  $N$  queens on an  $N \times N$  chessboard so that no two queens attack each other. **Pruning:** Instead of checking the whole board, we use boolean arrays to instantly check if a column or diagonal is already under attack.

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5

```

```

6 int total_n_queens_solutions = 0;
7
8 // Time Complexity: O(N!) in the worst case, but heavily pruned.
9 // Space Complexity: O(N) for the recursion stack and tracking arrays.
10 void solveNQueens(int row, int n, vector<bool>& col, vector<bool>& diag1,
    vector<bool>& diag2) {
11     // 1. BASE CASE: If we successfully placed a queen in all rows, we found a
        solution.
12     if (row == n) {
13         total_n_queens_solutions++;
14         return;
15     }
16
17     // 2. ITERATE CHOICES: Try placing a queen in every column of the current
        row.
18     for (int c = 0; c < n; ++c) {
19         // 3. PRUNING: Check if this column or either diagonal is already
            occupied.
20         // diag1 goes from top-left to bottom-right (constant row - col)
21         // diag2 goes from bottom-left to top-right (constant row + col)
22         if (col[c] || diag1[row - c + n - 1] || diag2[row + c]) {
23             continue;
24         }
25
26         // 4. DO: Place the queen and mark the column/diagonals as attacked.
27         col[c] = diag1[row - c + n - 1] = diag2[row + c] = true;
28
29         // 5. RECURSE: Move to the next row.
30         solveNQueens(row + 1, n, col, diag1, diag2);
31
32         // 6. UNDO: Remove the queen to backtrack and try the next column.
33         col[c] = diag1[row - c + n - 1] = diag2[row + c] = false;
34     }
35 }
36
37 int main() {
38     int n;
39     cin >> n;
40
41     vector<bool> col(n, false);
42     vector<bool> diag1(2 * n - 1, false);
43     vector<bool> diag2(2 * n - 1, false);
44
45     solveNQueens(0, n, col, diag1, diag2);
46     cout << total_n_queens_solutions << "\n";
47
48     return 0;
49 }

```

## 4. Greedy Fundamentals & Sorting

### 4.1. Greedy Heuristics: The Knapsack Problem

```

1 #include <iostream>

```

```

2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 struct Item {
8     int value;
9     int weight;
10 };
11
12 // Time Complexity: O(1)
13 bool compareByRatio(const Item& a, const Item& b) {
14     double ratio_a = (double)a.value / a.weight;
15     double ratio_b = (double)b.value / b.weight;
16     return ratio_a > ratio_b;
17 }
18
19 double getMaximumKnapsackValue(vector<Item>& items, int capacity) {
20     // Time Complexity: O(N log N) to sort the items
21     // Space Complexity: O(log N) auxiliary space for std::sort
22     sort(items.begin(), items.end(), compareByRatio);
23     double total_value = 0.0;
24     int current_weight = 0;
25
26     // Time Complexity: O(N) - Linear scan through the sorted items
27     for (const Item& item : items) {
28         if (current_weight + item.weight <= capacity) {
29             current_weight += item.weight;
30             total_value += item.value;
31         } else {
32             int remaining_capacity = capacity - current_weight;
33             total_value += item.value * ((double)remaining_capacity / item.
                weight);
34             break;
35         }
36     }
37     return total_value;
38 }
39
40 int main() {
41     int num_items, capacity;
42     cin >> num_items >> capacity;
43     // Space Complexity: O(N) where N is num_items
44     vector<Item> items(num_items);
45     // Time Complexity: O(N) to read items
46     for (int i = 0; i < num_items; ++i) {
47         cin >> items[i].value >> items[i].weight;
48     }
49
50     cout << getMaximumKnapsackValue(items, capacity) << "\n";
51     return 0;
52 }

```

## 4.2. Shopaholic [Kattis]

**Concept:** Greedy Sorting. To maximize the discount, sort items descending and get the 3rd, 6th, 9th, etc., items for free.

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  long long calculateMaximumDiscount(vector<long long>& prices) {
8      // Time Complexity: O(N log N)
9      sort(prices.rbegin(), prices.rend()); // Sort descending
10
11     long long total_discount = 0;
12     // Time Complexity: O(N)
13     for (size_t i = 2; i < prices.size(); i += 3) {
14         total_discount += prices[i];
15     }
16
17     return total_discount;
18 }
19
20 int main() {
21     int num_items;
22     cin >> num_items;
23
24     // Space Complexity: O(N)
25     vector<long long> prices(num_items);
26     for (int i = 0; i < num_items; ++i) {
27         cin >> prices[i];
28     }
29
30     cout << calculateMaximumDiscount(prices) << "\n";
31     return 0;
32 }

```

## 4.3. Frosh Week (Inversion Counting) [Kattis]

**Concept:** This problem asks for the minimum number of swaps to sort an array, which is exactly the number of "inversions". We use Merge Sort to count these efficiently.

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  // Time Complexity: O(N log N) total for the merge sort process
6  // Space Complexity: O(N) for the temporary array
7  long long mergeAndCount(vector<int>& array, vector<int>& temp, int left, int
mid, int right) {
8      int i = left;
9      int j = mid + 1;
10     int k = left;
11     long long inversions = 0;
12     while (i <= mid && j <= right) {

```

```

13         if (array[i] <= array[j]) {
14             temp[k++] = array[i++];
15         } else {
16             temp[k++] = array[j++];
17             // If array[i] > array[j], all remaining elements in the left half
18             // are also > array[j]
19             inversions += (mid - i + 1);
20         }
21     }
22     while (i <= mid) temp[k++] = array[i++];
23     while (j <= right) temp[k++] = array[j++];
24     for (i = left; i <= right; i++) array[i] = temp[i];
25
26     return inversions;
27 }
28
29 long long countSwaps(vector<int>& array, vector<int>& temp, int left, int right
) {
30     long long inversions = 0;
31     if (left < right) {
32         int mid = left + (right - left) / 2;
33         inversions += countSwaps(array, temp, left, mid);
34         inversions += countSwaps(array, temp, mid + 1, right);
35         inversions += mergeAndCount(array, temp, left, mid, right);
36     }
37     return inversions;
38 }
39
40 int main() {
41     int num_students;
42     if (!(cin >> num_students)) return 0;
43
44     vector<int> student_numbers(num_students);
45     vector<int> temp_array(num_students);
46     for (int i = 0; i < num_students; ++i) {
47         cin >> student_numbers[i];
48     }
49
50     cout << countSwaps(student_numbers, temp_array, 0, num_students - 1) << "\n
";
51     return 0;
52 }

```

## 5. Interval Scheduling & Placement

### 5.1. Greedy Scheduling (Activity Selection)

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6

```

```

7 struct Activity {
8     int start_time;
9     int end_time;
10 };
11
12 // Time Complexity: O(1) per comparison
13 bool compareActivities(Activity a, Activity b) {
14     return a.end_time < b.end_time;
15 }
16
17 int getMaximumActivities(vector<Activity>& activity_list) {
18     if (activity_list.empty()) return 0;
19     // Time Complexity: O(N log N) where N is the number of activities
20     // Space Complexity: O(log N) auxiliary space for std::sort (IntroSort)
21     sort(activity_list.begin(), activity_list.end(), compareActivities);
22     int count_selected = 1;
23     int current_end_time = activity_list[0].end_time;
24
25     // Time Complexity: O(N) - Single pass through the sorted activities
26     for (size_t i = 1; i < activity_list.size(); ++i) {
27         if (activity_list[i].start_time >= current_end_time) {
28             count_selected++;
29             current_end_time = activity_list[i].end_time;
30         }
31     }
32     return count_selected;
33 }
34
35 int main() {
36     int num_activities;
37     cin >> num_activities;
38     // Space Complexity: O(N) where N is num_activities
39     vector<Activity> activity_list(num_activities);
40     // Time Complexity: O(N) to read inputs
41     for (int i = 0; i < num_activities; ++i) {
42         cin >> activity_list[i].start_time >> activity_list[i].end_time;
43     }
44
45     cout << getMaximumActivities(activity_list) << "\n";
46     return 0;
47 }

```

## 5.2. Assigning Workstations [Kattis]

**Concept:** Greedy Scheduling. Sort researchers by arrival time. Use a Min-Heap to track when currently used workstations will lock.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 #include <queue>
5
6 using namespace std;
7 struct Researcher {
8     long long arrival;

```

```

9     long long stay;
10 };
11 // Time Complexity: O(1)
12 bool compareArrivals(const Researcher& a, const Researcher& b) {
13     return a.arrival < b.arrival;
14 }
15
16 int calculateSavedUnlocks(vector<Researcher>& researchers, long long
    lock_timeout) {
17     // Time Complexity: O(N log N)
18     sort(researchers.begin(), researchers.end(), compareArrivals);
19     // Space Complexity: O(N) in worst case
20     priority_queue<long long, vector<long long>, greater<long long>>
        active_machines;
21     int saved_unlocks = 0;
22
23     // Time Complexity: O(N log N) - N iterations, heap operations take log N
24     for (const Researcher& person : researchers) {
25         long long current_time = person.arrival;
26         // Remove machines that have already locked
27         while (!active_machines.empty() && active_machines.top() + lock_timeout
            < current_time) {
28             active_machines.pop();
29         }
30
31         // If a machine is available and hasn't locked
32         if (!active_machines.empty() && active_machines.top() <= current_time)
            {
33             saved_unlocks++;
34             active_machines.pop();
35         }
36
37         // Machine will be free after they finish
38         active_machines.push(current_time + person.stay);
39     }
40
41     return saved_unlocks;
42 }
43
44 int main() {
45     int num_researchers;
46     long long lock_timeout;
47     cin >> num_researchers >> lock_timeout;
48
49     vector<Researcher> researchers(num_researchers);
50     for (int i = 0; i < num_researchers; ++i) {
51         cin >> researchers[i].arrival >> researchers[i].stay;
52     }
53
54     cout << calculateSavedUnlocks(researchers, lock_timeout) << "\n";
55     return 0;
56 }

```



### 5.3. Birds on a Wire [Kattis]

**Concept:** Greedy Placement. Sort existing birds, then calculate how many fit in the gaps between them, including the edges.

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int getAdditionalBirds(long long wire_length, long long distance, vector<long
    long>& birds) {
8      if (wire_length <= 12) return 0;
9      // Must be at least 6 from each pole
10
11     // If no birds, calculate max possible on the whole valid wire
12     if (birds.empty()) {
13         return ((wire_length - 12) / distance) + 1;
14     }
15
16     // Time Complexity: O(N log N)
17     // Space Complexity: O(log N) auxiliary
18     sort(birds.begin(), birds.end());
19     int new_birds = 0;
20
21     // Check gap from left pole (requires 6 units)
22     if (birds[0] - 6 >= distance) {
23         new_birds += (birds[0] - 6) / distance;
24     }
25
26     // Check gaps between existing birds. Time Complexity: O(N)
27     for (size_t i = 0; i < birds.size() - 1; ++i) {
28         long long gap = birds[i+1] - birds[i];
29         if (gap >= 2 * distance) {
30             new_birds += (gap / distance) - 1;
31         }
32     }
33
34     // Check gap to right pole (requires 6 units)
35     long long last_bird = birds.back();
36     if ((wire_length - 6) - last_bird >= distance) {
37         new_birds += ((wire_length - 6) - last_bird) / distance;
38     }
39
40     return new_birds;
41 }
42
43 int main() {
44     long long wire_length, distance;
45     int num_birds;
46     cin >> wire_length >> distance >> num_birds;
47
48     vector<long long> birds(num_birds);
49     for (int i = 0; i < num_birds; ++i) {
50         cin >> birds[i];

```

```

51     }
52
53     cout << getAdditionalBirds(wire_length, distance, birds) << "\n";
54     return 0;
55 }

```

## 6. Priority Queues & Data Structures

### 6.1. Huffman Codes

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6
7  struct Node {
8      char character;
9      int frequency;
10     Node* left_child;
11     Node* right_child;
12 };
13
14 // Time Complexity: O(1) per comparison
15 struct CompareNodes {
16     bool operator()(Node* a, Node* b) {
17         return a->frequency > b->frequency;
18     }
19 };
20
21 Node* buildHuffmanTree(vector<char>& characters, vector<int>& frequencies) {
22     // Space Complexity: O(N) where N is the number of unique characters
23     priority_queue<Node*, vector<Node*>, CompareNodes> min_heap;
24     // Time Complexity: O(N log N) - N insertions, each taking O(log N) time
25     for (size_t i = 0; i < characters.size(); ++i) {
26         // Space Complexity: O(1) per node, O(N) total across the loop
27         Node* new_node = new Node{characters[i], frequencies[i], nullptr,
            nullptr};
28         min_heap.push(new_node);
29     }
30
31     // Time Complexity: O(N log N) - (N-1) merges, each doing O(log N) heap
        operations
32     while (min_heap.size() > 1) {
33         Node* left = min_heap.top();
34         min_heap.pop();
35         Node* right = min_heap.top(); min_heap.pop();
36
37         int combined_freq = left->frequency + right->frequency;
38         // Space Complexity: O(1) per internal node, O(N) total for N-1
            internal nodes
39         Node* parent = new Node{'\0', combined_freq, left, right};
40         min_heap.push(parent);
41     }

```

```

42
43     return min_heap.top();
44 }
45
46 int main() {
47     int num_chars;
48     cin >> num_chars;
49
50     // Space Complexity: O(N) for characters array
51     vector<char> characters(num_chars);
52     // Space Complexity: O(N) for frequencies array
53     vector<int> frequencies(num_chars);
54     // Time Complexity: O(N) to read inputs
55     for (int i = 0; i < num_chars; ++i) {
56         cin >> characters[i] >> frequencies[i];
57     }
58
59     Node* root = buildHuffmanTree(characters, frequencies);
60     return 0;
61 }

```

## 6.2. Boiling Vegetables [Kattis]

**Concept:** Priority Queue. Always slice the piece that is currently the largest. Keep track of the minimum piece globally to check the ratio.

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6  struct Veggie {
7      double original_weight;
8      int cuts;
9      double current_weight;
10 };
11 // Max-Heap comparator: order by current weight descending
12 struct CompareVeggie {
13     bool operator()(const Veggie& a, const Veggie& b) {
14         return a.current_weight < b.current_weight;
15     }
16 };
17
18 int getMinimumCuts(double target_ratio, vector<double>& weights) {
19     // Space Complexity: O(N)
20     priority_queue<Veggie, vector<Veggie>, CompareVeggie> max_heap;
21     double current_min_weight = 1e9;
22
23     // Time Complexity: O(N log N) to build
24     for (double w : weights) {
25         max_heap.push({w, 1, w});
26         if (w < current_min_weight) {
27             current_min_weight = w;
28         }
29     }

```

```

30     int total_cuts = 0;
31     // Time Complexity: O(Cuts * log N). Exact bound depends on inputs.
32     while (true) {
33         Veggie largest = max_heap.top();
34         // If the ratio between smallest and largest is valid, we are done
35         if (current_min_weight / largest.current_weight > target_ratio) {
36             break;
37         }
38
39         max_heap.pop();
40         largest.cuts++;
41         largest.current_weight = largest.original_weight / largest.cuts;
42
43         // Update global min if the new piece is the smallest
44         if (largest.current_weight < current_min_weight) {
45             current_min_weight = largest.current_weight;
46         }
47
48         max_heap.push(largest);
49         total_cuts++;
50     }
51
52     return total_cuts;
53 }
54
55 int main() {
56     double target_ratio;
57     int num_veggies;
58     cin >> target_ratio >> num_veggies;
59
60     vector<double> weights(num_veggies);
61     for (int i = 0; i < num_veggies; ++i) {
62         cin >> weights[i];
63     }
64
65     cout << getMinimumCuts(target_ratio, weights) << "\n";
66     return 0;
67 }
68 }

```

## 7. Minimum Spanning Trees (MST)

### 7.1. Prim's Algorithm (Adjacency Matrix)

```

1  #include <iostream>
2  #include <vector>
3  #include <climits>
4
5  using namespace std;
6  int getMinimumVertex(const vector<int>& min_edge_weights, const vector<bool>&
7      is_in_mst, int num_vertices) {
8      int min_weight = INT_MAX;
9      int min_vertex = -1;

```

```

10 // Time Complexity:  $O(V)$  - Must scan all vertices to find the minimum
11 for (int v = 0; v < num_vertices; ++v) {
12     if (!is_in_mst[v] && min_edge_weights[v] < min_weight) {
13         min_weight = min_edge_weights[v];
14         min_vertex = v;
15     }
16 }
17 return min_vertex;
18 }
19
20 int calculatePrimMSTMatrix(const vector<vector<int>>& graph_matrix, int
num_vertices) {
21     // Space Complexity:  $O(V)$  where  $V$  is num_vertices
22     vector<int> min_edge_weights(num_vertices, INT_MAX);
23     // Space Complexity:  $O(V)$ 
24     vector<bool> is_in_mst(num_vertices, false);
25
26     int total_mst_weight = 0;
27     min_edge_weights[0] = 0;
28     // Time Complexity:  $O(V^2)$  total. Outer loop runs  $V$  times.
29     for (int count = 0; count < num_vertices; ++count) {
30         // Inner operation 1:  $O(V)$  to find the minimum vertex
31         int current_u = getMinimumVertex(min_edge_weights, is_in_mst,
num_vertices);
32         is_in_mst[current_u] = true;
33         total_mst_weight += min_edge_weights[current_u];
34
35         // Inner operation 2:  $O(V)$  to update adjacent vertices
36         for (int v = 0; v < num_vertices; ++v) {
37             int edge_weight = graph_matrix[current_u][v];
38             if (edge_weight != 0 && !is_in_mst[v] && edge_weight <
min_edge_weights[v]) {
39                 min_edge_weights[v] = edge_weight;
40             }
41         }
42     }
43     return total_mst_weight;
44 }
45
46 int main() {
47     int num_vertices;
48     cin >> num_vertices;
49     // Space Complexity:  $O(V^2)$  for the adjacency matrix
50     vector<vector<int>> graph_matrix(num_vertices, vector<int>(num_vertices));
51     // Time Complexity:  $O(V^2)$  to read the matrix
52     for (int i = 0; i < num_vertices; ++i) {
53         for (int j = 0; j < num_vertices; ++j) {
54             cin >> graph_matrix[i][j];
55         }
56     }
57
58     cout << calculatePrimMSTMatrix(graph_matrix, num_vertices) << "\n";
59     return 0;
60 }

```

## 7.2. Prim's Algorithm (Priority Queue and Adjacency List)

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 using namespace std;
6
7 struct Edge {
8     int target_vertex;
9     int weight;
10 };
11
12 // Time Complexity:  $O(1)$  per comparison
13 struct CompareEdge {
14     bool operator()(const Edge& a, const Edge& b) {
15         return a.weight > b.weight;
16     }
17 };
18
19 int calculatePrimMSTList(const vector<vector<Edge>>& adjacency_list, int
num_vertices) {
20     // Space Complexity:  $O(E)$  in the worst case, where  $E$  is the number of edges
21     priority_queue<Edge, vector<Edge>, CompareEdge> min_heap;
22     // Space Complexity:  $O(V)$ 
23     vector<bool> is_in_mst(num_vertices, false);
24
25     int total_mst_weight = 0;
26     int edges_added = 0;
27
28     min_heap.push({0, 0});
29     // Time Complexity:  $O(E \log E)$  or equivalently  $O(E \log V)$ 
30     // The while loop processes at most  $E$  edges.
31     while (!min_heap.empty() && edges_added < num_vertices) {
32         // Heap pop takes  $O(\log E)$  time
33         Edge current_edge = min_heap.top();
34         min_heap.pop();
35
36         int current_vertex = current_edge.target_vertex;
37
38         if (is_in_mst[current_vertex]) continue;
39
40         is_in_mst[current_vertex] = true;
41         total_mst_weight += current_edge.weight;
42         edges_added++;
43
44         // Push unvisited neighbors. Over the whole algorithm, this runs  $E$ 
times total.
45         // Heap push takes  $O(\log E)$  time.
46         for (const Edge& neighbor : adjacency_list[current_vertex]) {
47             if (!is_in_mst[neighbor.target_vertex]) {
48                 min_heap.push(neighbor);
49             }
50         }
51     }

```

```

52     return total_mst_weight;
53 }
54
55 int main() {
56     int num_vertices, num_edges;
57     cin >> num_vertices >> num_edges;
58     // Space Complexity:  $O(V + E)$  for the adjacency list representation
59     vector<vector<Edge>> adjacency_list(num_vertices);
60     // Time Complexity:  $O(E)$  to read edges
61     for (int i = 0; i < num_edges; ++i) {
62         int u, v, weight;
63         cin >> u >> v >> weight;
64         adjacency_list[u].push_back({v, weight});
65         adjacency_list[v].push_back({u, weight});
66     }
67
68     cout << calculatePrimMSTList(adjacency_list, num_vertices) << "\n";
69     return 0;
70 }

```

### 7.3. Kruskal's Algorithm

#### 7.3.1 Bases for Kruskal's Algorithm

- **The Strategy:** Sort all edges in non-decreasing order of their weight. Pick the smallest edge. If it forms a cycle with the spanning tree formed so far, discard it. If not, include it.
- **Cycle Detection:** Uses the Disjoint Set Union (DSU) data structure. If both vertices of an edge belong to the same set, adding the edge will create a cycle.
- **Complexity:**  $O(E \log E)$  or  $O(E \log V)$ . It is generally preferred for sparse graphs where edges can be easily sorted.

#### 7.3.2 Implementation: Union-Find and Edge Sorting

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  struct Edge {
8      int source;
9      int destination;
10     int weight;
11 };
12
13 // Time Complexity:  $O(1)$ 
14 bool compareEdges(const Edge& a, const Edge& b) {
15     return a.weight < b.weight;
16 }
17
18 // --- Disjoint Set Union (DSU) Helper Functions ---
19

```

```

20 // Time Complexity:  $O(\alpha(V)) \sim O(1)$  due to path compression
21 int findParent(int vertex, vector<int>& parent_array) {
22     if (parent_array[vertex] == vertex) {
23         return vertex;
24     }
25     // Path compression: attach the node directly to the root
26     return parent_array[vertex] = findParent(parent_array[vertex], parent_array);
27 }
28
29 // Time Complexity:  $O(\alpha(V)) \sim O(1)$ 
30 void unionSets(int vertex_u, int vertex_v, vector<int>& parent_array, vector<int>& rank_array) {
31     int root_u = findParent(vertex_u, parent_array);
32     int root_v = findParent(vertex_v, parent_array);
33
34     if (root_u != root_v) {
35         // Union by rank: attach smaller tree under root of deeper tree
36         if (rank_array[root_u] < rank_array[root_v]) {
37             parent_array[root_u] = root_v;
38         } else if (rank_array[root_u] > rank_array[root_v]) {
39             parent_array[root_v] = root_u;
40         } else {
41             parent_array[root_v] = root_u;
42             rank_array[root_u]++;
43         }
44     }
45 }
46
47 // --- Main Kruskal Logic ---
48
49 int calculateKruskalMST(vector<Edge>& edge_list, int num_vertices) {
50     // Time Complexity:  $O(E \log E)$  to sort the edges
51     // Space Complexity:  $O(\log E)$  auxiliary space for std::sort
52     sort(edge_list.begin(), edge_list.end(), compareEdges);
53     // Space Complexity:  $O(V)$  for parent and rank arrays
54     vector<int> parent_array(num_vertices);
55     vector<int> rank_array(num_vertices, 0);
56     // Initialize DSU: Every vertex is its own parent initially
57     for (int i = 0; i < num_vertices; ++i) {
58         parent_array[i] = i;
59     }
60
61     int total_mst_weight = 0;
62     int edges_added = 0;
63     // Time Complexity:  $O(E * \alpha(V)) \sim O(E)$  for the loop
64     for (const Edge& edge : edge_list) {
65         // Stop early if we've added V-1 edges (a complete spanning tree)
66         if (edges_added == num_vertices - 1) break;
67         int root_source = findParent(edge.source, parent_array);
68         int root_dest = findParent(edge.destination, parent_array);
69         // If they don't share a parent, adding this edge won't create a cycle
70         if (root_source != root_dest) {
71             total_mst_weight += edge.weight;
72             unionSets(root_source, root_dest, parent_array, rank_array);

```

```

73         edges_added++;
74     }
75 }
76
77 return total_mst_weight;
78 }
79
80 int main() {
81     int num_vertices, num_edges;
82     cin >> num_vertices >> num_edges;
83     // Space Complexity: O(E) to store the list of all edges
84     vector<Edge> edge_list(num_edges);
85     // Time Complexity: O(E) to read edges
86     for (int i = 0; i < num_edges; ++i) {
87         cin >> edge_list[i].source >> edge_list[i].destination >> edge_list[i].
            weight;
88     }
89
90     cout << calculateKruskalMST(edge_list, num_vertices) << "\n";
91     return 0;
92 }

```

#### 7.4. Lost Map [Kattis]

**Concept:** Kruskal's MST from an Adjacency Matrix. The shortest paths between adjacent villages form the MST of the complete graph.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6 struct Edge {
7     int u, v, weight;
8 };
9 bool compareEdges(const Edge& a, const Edge& b) {
10     return a.weight < b.weight;
11 }
12
13 // Time Complexity: O(1) amortized
14 int findParent(int i, vector<int>& parent) {
15     if (parent[i] == i) return i;
16     return parent[i] = findParent(parent[i], parent);
17 }
18
19 void printLostMapEdges(vector<Edge>& edges, int num_vertices) {
20     // Time Complexity: O(V^2 log(V^2)) to sort edges
21     sort(edges.begin(), edges.end(), compareEdges);
22     // Space Complexity: O(V)
23     vector<int> parent(num_vertices + 1);
24     for (int i = 1; i <= num_vertices; ++i) parent[i] = i;
25
26     int edges_printed = 0;
27     // Time Complexity: O(V^2) loop iterations
28     for (const Edge& e : edges) {

```

```

29         if (edges_printed == num_vertices - 1) break;
30         int root_u = findParent(e.u, parent);
31         int root_v = findParent(e.v, parent);
32         if (root_u != root_v) {
33             cout << e.u << " " << e.v << "\n";
34             parent[root_u] = root_v;
35             edges_printed++;
36         }
37     }
38 }
39
40 int main() {
41     int num_vertices;
42     cin >> num_vertices;
43     // Space Complexity: O(V^2)
44     vector<Edge> edges;
45     for (int i = 1; i <= num_vertices; ++i) {
46         for (int j = 1; j <= num_vertices; ++j) {
47             int weight;
48             cin >> weight;
49             // Only add upper triangle of the matrix to avoid duplicates
50             if (i < j) {
51                 edges.push_back({i, j, weight});
52             }
53         }
54     }
55
56     printLostMapEdges(edges, num_vertices);
57     return 0;
58 }

```

## 8. Ad-Hoc / Mathematical Greedy

### 8.1. Ants [Kattis]

**Concept:** Independent movement. When ants collide, they bounce, which is mathematically identical to them passing through each other. Min time: The last ant to fall off if they all walk to their \*closest\* edge. Max time: The last ant to fall off if they all walk to their \*furthest\* edge.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 void calculateAntTimes(int pole_length, const vector<int>& ant_positions) {
8     int earliest_time = 0;
9     int latest_time = 0;
10
11     // Time Complexity: O(N) - single pass
12     for (int pos : ant_positions) {
13         int distance_left = pos;
14         int distance_right = pole_length - pos;

```

```

15         int min_dist = min(distance_left, distance_right);
16         int max_dist = max(distance_left, distance_right);
17         if (min_dist > earliest_time) earliest_time = min_dist;
18         if (max_dist > latest_time) latest_time = max_dist;
19     }
20
21     cout << earliest_time << " " << latest_time << "\n";
22 }
23
24 int main() {
25     int num_test_cases;
26     cin >> num_test_cases;
27     while (num_test_cases--) {
28         int pole_length, num_ants;
29         cin >> pole_length >> num_ants;
30         // Space Complexity: O(N)
31         vector<int> ant_positions(num_ants);
32         for (int i = 0; i < num_ants; ++i) {
33             cin >> ant_positions[i];
34         }
35
36         calculateAntTimes(pole_length, ant_positions);
37     }
38     return 0;
39 }
40

```

## 9. Strategies and Base Concepts

### 9.1. Dynamic Programming: General Strategy

Dynamic Programming (DP) is a systematic way to avoid doing the same work twice. If a problem feels like magic, it is usually just a well-defined DP state.

#### 9.1.1 Step 1: Identifying a DP Problem

Do not waste time on DP if a simple Greedy approach works. A problem requires DP if it triggers these conditions:

- **The Goal:** The problem asks for an extreme value (e.g., “*maximize profit*”, “*minimize cost*”) or an exhaustive count (e.g., “*how many ways to...*”).
- **Overlapping Subproblems:** If you draw out the decision tree and see that you are calculating the exact same scenario multiple times, you need DP to memorize those answers.
- **Optimal Substructure:** The absolute best solution for the large problem can be built simply by combining the absolute best solutions of smaller sub-problems.

#### 9.1.2 Step 2: Connecting the Dots

Once your state is minimally defined, the rest of the algorithm naturally follows:

- **Base Cases:** The trivial “bottom” of the problem (e.g., If my knapsack capacity is 0, my maximum value is 0).

- **State Transition:** The mathematical formula that tells you how to calculate the current state from previous states. It usually looks like a `max()` or `min()` function choosing between two paths (e.g., “*Do I take this item, or do I leave it?*”).
- **Tabulation:** Setting up your nested for loops to fill your array from the base cases all the way up to the final answer (the bottom-up approach).

### 9.2. Sequence Alignment (Edit Distance)

**Concept:** Find the minimum cost to transform one string into another using insertions, deletions, and substitutions.

**State:** `dp[i][j]` represents the minimum cost to align the first `i` characters of string 1 with the first `j` characters of string 2.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <algorithm>
5
6  using namespace std;
7
8  int getMinimumAlignmentCost(const string& str1, const string& str2, int
    insert_cost, int delete_cost, int replace_cost) {
9      int len1 = str1.length();
10     int len2 = str2.length();
11
12     // Space Complexity: O(N * M) where N and M are the lengths of the strings
13     vector<vector<int>> dp(len1 + 1, vector<int>(len2 + 1, 0));
14     // Base cases: transforming to an empty string requires all deletions or
        insertions
15     for (int i = 0; i <= len1; ++i) dp[i][0] = i * delete_cost;
16     for (int j = 0; j <= len2; ++j) dp[0][j] = j * insert_cost;
17     // Time Complexity: O(N * M) to fill the table
18     for (int i = 1; i <= len1; ++i) {
19         for (int j = 1; j <= len2; ++j) {
20             if (str1[i - 1] == str2[j - 1]) {
21                 // Characters match, no additional cost
22                 dp[i][j] = dp[i - 1][j - 1];
23             } else {
24                 // Characters differ, take the minimum of the three operations
25                 int cost_replace = dp[i - 1][j - 1] + replace_cost;
26                 int cost_delete = dp[i - 1][j] + delete_cost;
27                 int cost_insert = dp[i][j - 1] + insert_cost;
28                 dp[i][j] = min({cost_replace, cost_delete, cost_insert});
29             }
30         }
31     }
32     return dp[len1][len2];
33 }

```

### 9.3. Optimal Binary Search Tree (OBST)

**Concept:** Given a sorted array of keys and their search frequencies, construct a BST that minimizes the total expected search cost.

**State:**  $dp[i][j]$  represents the minimum search cost for a BST containing keys from index  $i$  to  $j$ .

```

1  #include <iostream>
2  #include <vector>
3  #include <climits>
4  #include <algorithm>
5
6  using namespace std;
7
8  // Helper function to get the sum of frequencies from index i to j
9  // Time Complexity: O(1) because we use a precomputed prefix sum array
10 int getFrequencySum(const vector<int>& prefix_sums, int i, int j) {
11     if (i == 0) return prefix_sums[j];
12     return prefix_sums[j] - prefix_sums[i - 1];
13 }
14
15 int calculateOBSTCost(const vector<int>& frequencies, int num_keys) {
16     // Space Complexity: O(N^2) for the DP table
17     vector<vector<int>>> dp(num_keys, vector<int>(num_keys, 0));
18     // Space Complexity: O(N) for prefix sums
19     vector<int> prefix_sums(num_keys, 0);
20     prefix_sums[0] = frequencies[0];
21     for (int i = 1; i < num_keys; ++i) {
22         prefix_sums[i] = prefix_sums[i - 1] + frequencies[i];
23     }
24
25     // Base Case: Trees with a single key have a cost equal to their frequency
26     for (int i = 0; i < num_keys; ++i) {
27         dp[i][i] = frequencies[i];
28     }
29
30     // Time Complexity: O(N^3) total.
31     // L is the length of the chain being evaluated (from 2 to num_keys)
32     for (int L = 2; L <= num_keys; ++L) {
33         for (int i = 0; i <= num_keys - L; ++i) {
34             int j = i + L - 1;
35             dp[i][j] = INT_MAX;
36
37             // Try making every key 'k' in the range [i, j] the root
38             for (int k = i; k <= j; ++k) {
39                 // Cost of left subtree (0 if k is the first element)
40                 int left_cost = (k > i) ? dp[i][k - 1] : 0;
41                 // Cost of right subtree (0 if k is the last element)
42                 int right_cost = (k < j) ? dp[k + 1][j] : 0;
43
44                 // Total cost = cost of subtrees + sum of frequencies in this
45                 // range
46                 // (because pushing everything down one level adds 1 *
47                 // frequency to the total cost)
48                 int total_cost = left_cost + right_cost + getFrequencySum(
49                     prefix_sums, i, j);
50                 dp[i][j] = min(dp[i][j], total_cost);
51             }
52         }
53     }
54 }
```

```

50     }
51     return dp[0][num_keys - 1];
52 }
```

## 10. Combinatorics and Coin Change

### 10.1. Dice Combinations [CSES]

**Concept:** Find the number of ways to construct sum  $n$  throwing a 6-sided dice.

**State:**  $dp[i]$  = number of ways to make sum  $i$ .

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  #define int long long
6  const int MOD = 1e9 + 7;
7
8  int countDiceWays(int target_sum) {
9      // Space: O(N)
10     vector<int> dp(target_sum + 1, 0);
11     dp[0] = 1; // 1 way to make sum 0 (throw nothing)
12
13     // Time: O(N * 6) -> O(N)
14     for (int i = 1; i <= target_sum; ++i) {
15         for (int dice = 1; dice <= 6; ++dice) {
16             if (i - dice >= 0) {
17                 dp[i] = (dp[i] + dp[i - dice]) % MOD;
18             }
19         }
20     }
21     return dp[target_sum];
22 }
23
24 int32_t main() {
25     int n; cin >> n;
26     cout << countDiceWays(n) << "\n";
27     return 0;
28 }
```

### 10.2. Minimizing Coins [CSES]

**Concept:** Find the minimum number of coins to make a target sum.

**State:**  $dp[i]$  = minimum coins to make sum  $i$ .

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6  #define int long long
7
8  int getMinCoins(int target, const vector<int>& coins) {
9      // Space: O(X) where X is target
10     vector<int> dp(target + 1, 1e9);
```



```

11     dp[0] = 0;
12
13     // Time: O(N * X)
14     for (int i = 1; i <= target; ++i) {
15         for (int coin : coins) {
16             if (i - coin >= 0) {
17                 dp[i] = min(dp[i], dp[i - coin] + 1);
18             }
19         }
20     }
21     return dp[target] == 1e9 ? -1 : dp[target];
22 }
23
24 int32_t main() {
25     int num_coins, target;
26     cin >> num_coins >> target;
27     vector<int> coins(num_coins);
28     // Space: O(N)
29     for (int i = 0; i < num_coins; ++i) cin >> coins[i];
30     cout << getMinCoins(target, coins) << "\n";
31     return 0;
32 }

```

### 10.3. Coin Combinations I & II [CSES]

**Concept:** *Coin Combinations I* cares about permutations (order matters). The outer loop is the target weight. *Coin Combinations II* cares about combinations (order DOES NOT matter). The outer loop is the coins array.

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  #define int long long
6  const int MOD = 1e9 + 7;
7
8  // --- COIN COMBINATIONS I (Permutations) ---
9  int countPermutations(int target, const vector<int>& coins) {
10     vector<int> dp(target + 1, 0);
11     // Space: O(X)
12     dp[0] = 1;
13     // Time: O(X * N)
14     for (int weight = 1; weight <= target; ++weight) {
15         for (int coin : coins) {
16             if (weight - coin >= 0) dp[weight] = (dp[weight] + dp[weight - coin]
17             ) % MOD;
18         }
19     }
20     return dp[target];
21 }
22
23 // --- COIN COMBINATIONS II (Combinations) ---
24 int countCombinations(int target, const vector<int>& coins) {
25     vector<int> dp(target + 1, 0);
26     // Space: O(X)

```

```

26     dp[0] = 1;
27     // Time: O(N * X) -> Notice the loops are swapped!
28     for (int coin : coins) {
29         for (int weight = 1; weight <= target; ++weight) {
30             if (weight - coin >= 0) dp[weight] = (dp[weight] + dp[weight - coin]
31             ) % MOD;
32         }
33     }
34     return dp[target];
35 }

```

### 10.4. Removing Digits [CSES]

**Concept:** Subtract the largest digit of a number until it becomes 0.

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int getMinSteps(int n) {
8     // Space: O(N)
9     vector<int> dp(n + 1, 1e9);
10     dp[0] = 0;
11
12     // Time: O(N * log10(N))
13     for (int i = 1; i <= n; ++i) {
14         int temp = i;
15         int max_digit = 0;
16         // Extract digits
17         while (temp > 0) {
18             max_digit = max(max_digit, temp % 10);
19             temp /= 10;
20         }
21         dp[i] = dp[i - max_digit] + 1;
22     }
23     return dp[n];
24 }
25
26 int main() {
27     int n; cin >> n;
28     cout << getMinSteps(n) << "\n";
29     return 0;
30 }

```

### 10.5. Problem B: Restaurant Orders [Kattis]

**Concept:** Unbounded Knapsack (Coin Change Combinations). We must track if a cost can be made in 0 ways, exactly 1 way, or > 1 ways (Ambiguous).

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;

```



```

5
6 void processOrders(int num_items, const vector<int>& costs, int num_orders,
   const vector<int>& queries) {
7     int max_order = 0;
8     for (int q : queries) max_order = max(max_order, q);
9
10    // Space: O(M) where M is the max order cost (<= 30000)
11    // dp array tracks NUMBER of ways to make a cost.
12    // We cap it at 2 to prevent overflow.
13    vector<int> dp(max_order + 1, 0);
14    vector<int> first_item_used(max_order + 1, -1);
15    dp[0] = 1;
16
17    // Time: O(N * M)
18    for (int i = 0; i < num_items; ++i) {
19        for (int w = costs[i]; w <= max_order; ++w) {
20            if (dp[w - costs[i]] > 0) {
21                dp[w] += dp[w - costs[i]];
22                if (dp[w] > 2) dp[w] = 2; // Cap at 2 (Ambiguous)
23
24                // If it's the first way we've found to make 'w', record the
                // item
25                if (first_item_used[w] == -1 || dp[w - costs[i]] == 1) {
26                    first_item_used[w] = i;
27                }
28            }
29        }
30    }
31
32    // Answer queries. Time: O(M) per query to backtrack
33    for (int target : queries) {
34        if (dp[target] == 0) {
35            cout << "Impossible\n";
36        } else if (dp[target] > 1) {
37            cout << "Ambiguous\n";
38        } else {
39            vector<int> receipt;
40            int curr = target;
41            while (curr > 0) {
42                int item_idx = first_item_used[curr];
43                receipt.push_back(item_idx + 1); // 1-based indexing
44                curr -= costs[item_idx];
45            }
46            for (int i = receipt.size() - 1; i >= 0; --i) {
47                cout << receipt[i] << " ";
48            }
49            cout << "\n";
50        }
51    }
52 }

```

## 11. Knapsack Variants

### 11.1. 0/1 Knapsack Problem

**Concept:** Maximize the value in a knapsack of limited capacity. You must take an item entirely or leave it (no fractions).

**State:**  $dp[i][w]$  is the maximum value using a subset of the first  $i$  items with a weight capacity limit of  $w$ .

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 struct Item {
8     int value;
9     int weight;
10 };
11
12 int calculate01Knapsack(int max_capacity, const vector<Item>& items) {
13     int num_items = items.size();
14     // Space Complexity: O(N * W) where N is items, W is capacity
15     vector<vector<int>> dp(num_items + 1, vector<int>(max_capacity + 1, 0));
16     // Time Complexity: O(N * W)
17     for (int i = 1; i <= num_items; ++i) {
18         for (int w = 0; w <= max_capacity; ++w) {
19
20             // Item's weight is based on 0-indexing
21             int current_weight = items[i - 1].weight;
22             int current_value = items[i - 1].value;
23
24             if (current_weight <= w) {
25                 // Choice: Max of (Leaving the item) OR (Taking the item +
26                 // value of remaining capacity)
27                 dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - current_weight] +
28                             current_value);
29             } else {
30                 // Too heavy, we MUST leave it
31                 dp[i][w] = dp[i - 1][w];
32             }
33         }
34     }
35     return dp[num_items][max_capacity];
36 }

```

### 11.2. Book Shop [CSES]

**Concept:** Classic 0/1 Knapsack. *CSES limits memory tightly, so a 1D Array is required, and int must be 32-bit to avoid TLE/MLE.*

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>

```

```

4
5 using namespace std;
6
7 int getMaxPages(int max_price, const vector<int>& prices, const vector<int>&
  pages) {
8     int num_books = prices.size();
9     // Space: O(X) where X is max_price. 1D Array to save memory!
10    vector<int> dp(max_price + 1, 0);
11    // Time: O(N * X)
12    for (int i = 0; i < num_books; ++i) {
13        // Must traverse backwards for 1D 0/1 Knapsack to avoid reusing the
          same item
14        for (int weight = max_price; weight >= prices[i]; --weight) {
15            dp[weight] = max(dp[weight], dp[weight - prices[i]] + pages[i]);
16        }
17    }
18    return dp[max_price];
19 }
20
21 int main() {
22     int num_books, max_price;
23     cin >> num_books >> max_price;
24
25     vector<int> prices(num_books); // Space: O(N)
26     vector<int> pages(num_books);
27     // Space: O(N)
28
29     for (int i = 0; i < num_books; ++i) cin >> prices[i];
30     for (int i = 0; i < num_books; ++i) cin >> pages[i];
31
32     cout << getMaxPages(max_price, prices, pages) << "\n";
33     return 0;
34 }

```

### 11.3. Money Sums & Two Sets II [CSES]

**Concept:** Subset Sum variants. *Money Sums* finds all possible sums. *Two Sets II* finds the number of ways to partition an array into two equal subsets.

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5 #define int long long
6 const int MOD = 1e9 + 7;
7
8 // --- MONEY SUMS ---
9 void printPossibleSums(int num_coins, const vector<int>& coins) {
10     int max_possible_sum = 0;
11     for (int c : coins) max_possible_sum += c;
12
13     // Space: O(Sum)
14     vector<bool> dp(max_possible_sum + 1, false);
15     dp[0] = true;
16

```

```

17     // Time: O(N * Sum)
18     for (int coin : coins) {
19         for (int w = max_possible_sum; w >= coin; --w) {
20             if (dp[w - coin]) dp[w] = true;
21         }
22     }
23
24     vector<int> valid_sums;
25     for (int i = 1; i <= max_possible_sum; ++i) {
26         if (dp[i]) valid_sums.push_back(i);
27     }
28
29     cout << valid_sums.size() << "\n";
30     for (int sum : valid_sums) cout << sum << " ";
31     cout << "\n";
32 }
33
34 // --- TWO SETS II ---
35 // Modular exponentiation for division under modulo
36 int modInverse(int base, int exp) {
37     int res = 1;
38     while (exp > 0) {
39         if (exp % 2 == 1) res = (res * base) % MOD;
40         base = (base * base) % MOD;
41         exp /= 2;
42     }
43     return res;
44 }
45
46 int countTwoSets(int n) {
47     int total_sum = n * (n + 1) / 2;
48     if (total_sum % 2 != 0) return 0; // Odd sum cannot be halved
49
50     int target = total_sum / 2;
51     // Space: O(Target)
52     vector<int> dp(target + 1, 0);
53     dp[0] = 1;
54     // Time: O(N * Target)
55     for (int i = 1; i <= n; ++i) {
56         for (int w = target; w >= i; --w) {
57             dp[w] = (dp[w] + dp[w - i]) % MOD;
58         }
59     }
60
61     // We calculated permutations counting both sets, must divide by 2
62     return (dp[target] * modInverse(2, MOD - 2)) % MOD;
63 }

```

### 11.4. Problem A: Knapsack [Kattis]

**Concept:** Classic 0/1 Knapsack, but requires reading until End of File (EOF) and backtracking the 2D DP matrix to print the indices of the chosen items.

```

1 #include <iostream>
2 #include <vector>

```

```

3  #include <algorithm>
4
5  using namespace std;
6
7  struct Item {
8      int value;
9      int weight;
10     int original_index;
11 };
12
13 void solveKnapsack(int capacity, int num_items, const vector<Item>& items) {
14     // Space: O(N * C)
15     vector<vector<int>> dp(num_items + 1, vector<int>(capacity + 1, 0));
16     // Time: O(N * C)
17     for (int i = 1; i <= num_items; ++i) {
18         for (int w = 0; w <= capacity; ++w) {
19             int current_weight = items[i - 1].weight;
20             int current_value = items[i - 1].value;
21
22             if (current_weight <= w) {
23                 dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - current_weight] +
24                                 current_value);
25             } else {
26                 dp[i][w] = dp[i - 1][w];
27             }
28         }
29     }
30
31     // Backtracking to find chosen items. Time: O(N)
32     vector<int> chosen_indices;
33     int current_capacity = capacity;
34     for (int i = num_items; i > 0; --i) {
35         // If value changed from the row above, we included this item
36         if (dp[i][current_capacity] != dp[i - 1][current_capacity]) {
37             chosen_indices.push_back(items[i - 1].original_index);
38             current_capacity -= items[i - 1].weight;
39         }
40     }
41
42     cout << chosen_indices.size() << "\n";
43     for (int idx : chosen_indices) {
44         cout << idx << " ";
45     }
46     cout << "\n";
47 }
48
49 int main() {
50     int capacity, num_items;
51     // Read until EOF
52     while (cin >> capacity >> num_items) {
53         vector<Item> items(num_items);
54         for (int i = 0; i < num_items; ++i) {
55             cin >> items[i].value >> items[i].weight;
56             items[i].original_index = i;
57         }
58     }
59 }

```

```

57     solveKnapsack(capacity, num_items, items);
58 }
59 return 0;
60 }

```

### 11.5. Problem C: Presidential Elections [Kattis]

**Concept:** Minimizing weight for a target value. Weight = "Voters needed to swing state", Value = "Electoral Delegates".

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6  #define int long long
7
8  struct State {
9      int delegates;
10     int required_voters_to_swing;
11 };
12
13 int32_t main() {
14     int num_states;
15     if (!(cin >> num_states)) return 0;
16
17     vector<State> valid_states;
18     int total_delegates = 0;
19     int guaranteed_delegates = 0;
20
21     for (int i = 0; i < num_states; ++i) {
22         int delegates, const_c, const_f, undecided;
23         cin >> delegates >> const_c >> const_f >> undecided;
24         total_delegates += delegates;
25
26         int total_voters = const_c + const_f + undecided;
27         int votes_needed_to_win = (total_voters / 2) + 1;
28
29         if (const_c >= votes_needed_to_win) {
30             guaranteed_delegates += delegates;
31             // Already won
32         } else {
33             int gap = votes_needed_to_win - const_c;
34             if (gap <= undecided) {
35                 // Winnable state
36                 valid_states.push_back({delegates, gap});
37             }
38         }
39     }
40
41     int delegates_needed = (total_delegates / 2) + 1;
42     int missing_delegates = delegates_needed - guaranteed_delegates;
43
44     if (missing_delegates <= 0) {
45         cout << 0 << "\n";
46     }
47 }

```

```

46     return 0;
47 }
48
49 // dp[v] = min voters needed to get EXACTLY v delegates. Space: O(Total
    Delegates)
50 vector<int> dp(total_delegates + 1, 1e15);
51 // Use 1e15 for infinity
52 dp[0] = 0;
53 // Time: O(N * D) where D is total delegates (<= 2016)
54 for (const State& s : valid_states) {
55     for (int v = total_delegates; v >= s.delegates; --v) {
56         if (dp[v - s.delegates] != 1e15) {
57             dp[v] = min(dp[v], dp[v - s.delegates] + s.
                required_voters_to_swing);
58         }
59     }
60 }
61
62 int min_total_voters = 1e15;
63 for (int v = missing_delegates; v <= total_delegates; ++v) {
64     min_total_voters = min(min_total_voters, dp[v]);
65 }
66
67 if (min_total_voters == 1e15) cout << "impossible\n";
68 else cout << min_total_voters << "\n";
69
70 return 0;
71 }

```

## 12. Jumps and Sequences 1D

### 12.1. Maximum Weight Independent Set (MWIS) on a Path

**Concept:** Also known as the "House Robber" problem. Find the maximum subset sum of an array such that no two selected elements are adjacent.

**State:** dp[i] represents the maximum weight independent set using a prefix of the first i elements.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int calculateMWIS(const vector<int>& weights) {
8     int num_elements = weights.size();
9     if (num_elements == 0) return 0;
10    if (num_elements == 1) return weights[0];
11    // Space Complexity: O(N) to store the DP table
12    // (Note: This can be optimized to O(1) by only tracking the last two
        values,
13    // but O(N) is strictly required if you need to backtrack to find the
        chosen items later).
14    vector<int> dp(num_elements, 0);

```

```

15    dp[0] = weights[0];
16    dp[1] = max(weights[0], weights[1]);
17    // Time Complexity: O(N) - Linear scan
18    for (int i = 2; i < num_elements; ++i) {
19        // Choice: Max of (Excluding the current element) OR (Including it +
        the max from i-2)
20        dp[i] = max(dp[i - 1], dp[i - 2] + weights[i]);
21    }
22
23    return dp[num_elements - 1];
24 }
25 }

```

### 12.2. Frog 1: Jump 1 or 2 Stones

**Concept:** Find the minimum cost to reach the last stone. You can jump 1 or 2 steps. Cost is the absolute difference in height.

```

1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <algorithm>
5
6 using namespace std;
7 #define int long long
8
9 int getMinCostFrog1(int num_stones, const vector<int>& heights) {
10    if (num_stones == 1) return 0;
11    // Space: O(N)
12    vector<int> dp(num_stones, 1e9);
13
14    // Base cases
15    dp[0] = 0;
16    dp[1] = abs(heights[1] - heights[0]);
17
18    // Time: O(N)
19    for (int i = 2; i < num_stones; ++i) {
20        int cost_from_prev1 = dp[i - 1] + abs(heights[i] - heights[i - 1]);
21        int cost_from_prev2 = dp[i - 2] + abs(heights[i] - heights[i - 2]);
22
23        dp[i] = min(cost_from_prev1, cost_from_prev2);
24    }
25
26    return dp[num_stones - 1];
27 }

```

### 12.3. Frog 2: Jump up to K Stones

**Concept:** A generalization of Frog 1. The frog can jump from any of the  $K$  previous stones.

```

1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4 #include <algorithm>

```

```

5
6 using namespace std;
7 #define int long long
8
9 int getMinCostFrog2(int num_stones, int max_jump, const vector<int>& heights) {
10     // Space: O(N)
11     vector<int> dp(num_stones, 1e9);
12     dp[0] = 0;
13
14     // Time: O(N * K)
15     for (int i = 1; i < num_stones; ++i) {
16         // Check all valid previous stones we could have jumped from
17         for (int jump = 1; jump <= max_jump; ++jump) {
18             if (i - jump >= 0) {
19                 int current_cost = dp[i - jump] + abs(heights[i] - heights[i -
20                     jump]);
21                 dp[i] = min(dp[i], current_cost);
22             }
23         }
24     }
25     return dp[num_stones - 1];
26 }

```

#### 12.4. Problem D: Spiderman's Workout [Kattis]

**Concept:** A 2D DP pathfinding problem. We must track the minimum \*maximum height\* reached, and strictly stay above height 0.

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5
6 using namespace std;
7
8 void solveSpiderman(int num_distances, const vector<int>& distances) {
9     int max_possible_height = 1000;
10    // Sum of all distances is <= 1000
11
12    // dp[i][h] = minimum max-height reached using first 'i' moves and ending
13    // at height 'h'
14    // Space: O(N * MaxSum)
15    vector<vector<int>> dp(num_distances + 1, vector<int>(max_possible_height +
16        1, 1e9));
17    vector<vector<char>> parent(num_distances + 1, vector<char>(
18        max_possible_height + 1, ' '));
19
20    dp[0][0] = 0;
21    // Time: O(N * MaxSum)
22    for (int i = 0; i < num_distances; ++i) {
23        int dist = distances[i];
24        for (int h = 0; h <= max_possible_height; ++h) {
25            if (dp[i][h] == 1e9) continue;
26            // Move UP

```

```

27            if (h + dist <= max_possible_height) {
28                int peak_if_up = max(dp[i][h], h + dist);
29                if (peak_if_up < dp[i + 1][h + dist]) {
30                    dp[i + 1][h + dist] = peak_if_up;
31                    parent[i + 1][h + dist] = 'U';
32                }
33            }
34
35            // Move DOWN (Must not go below ground)
36            if (h - dist >= 0) {
37                int peak_if_down = dp[i][h];
38                // Going down doesn't increase max height
39                if (peak_if_down < dp[i + 1][h - dist]) {
40                    dp[i + 1][h - dist] = peak_if_down;
41                    parent[i + 1][h - dist] = 'D';
42                }
43            }
44        }
45    }
46
47    // Must end exactly at ground level (0)
48    if (dp[num_distances][0] == 1e9) {
49        cout << "IMPOSSIBLE\n";
50        return;
51    }
52
53    // Backtrack to get moves
54    string path = "";
55    int current_h = 0;
56    for (int i = num_distances; i > 0; --i) {
57        char move = parent[i][current_h];
58        path += move;
59        if (move == 'U') current_h -= distances[i - 1];
60        else current_h += distances[i - 1];
61    }
62
63    reverse(path.begin(), path.end());
64    cout << path << "\n";
65 }

```

### 13. Grid, Matrices and Shapes

#### 13.1. Matrix Chain Multiplication (Interval DP)

**Concept:** Find the most efficient way to multiply a given sequence of matrices. **State:** Unlike standard DP that iterates linearly, Interval DP iterates by the *length of the interval* (L).  $dp[i][j]$  represents the minimum cost to multiply matrices from index  $i$  to  $j$ .

```

1 #include <iostream>
2 #include <vector>
3 #include <climits>
4
5 using namespace std;
6
7 // dims array represents matrix dimensions.

```

```

8 // Matrix A[i] has dimensions dims[i-1] x dims[i]
9 int matrixChainMultiplication(const vector<int>& dims) {
10     int num_matrices = dims.size() - 1;
11
12     // Space Complexity: O(N^2)
13     // dp[i][j] = min scalar multiplications to compute matrix A[i]...A[j]
14     vector<vector<int>> dp(num_matrices + 1, vector<int>(num_matrices + 1, 0));
15
16     // Time Complexity: O(N^3)
17     // Base case: L = 1 (single matrix) costs 0, already initialized.
18
19     // L is the length of the interval (chain of matrices) we are evaluating
20     for (int L = 2; L <= num_matrices; ++L) {
21         for (int i = 1; i <= num_matrices - L + 1; ++i) {
22             int j = i + L - 1;
23             dp[i][j] = INT_MAX;
24
25             // Try every possible split point 'k' between i and j-1
26             for (int k = i; k < j; ++k) {
27                 // Cost = Cost of Left Chain + Cost of Right Chain + Cost of
28                     // multiplying them together
29                 int current_cost = dp[i][k] + dp[k + 1][j] + (dims[i - 1] *
30                     dims[k] * dims[j]);
31                 dp[i][j] = min(dp[i][j], current_cost);
32             }
33         }
34     }
35
36     // Return the minimum cost to multiply the entire chain from 1 to N
37     return dp[1][num_matrices];
38 }

```

### 13.2. Grid Paths [CSES]

**Concept:** Count paths from top-left to bottom-right avoiding traps (\*).

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4
5 using namespace std;
6 #define int long long
7 const int MOD = 1e9 + 7;
8
9 int countGridPaths(int n, const vector<string>& grid) {
10     if (grid[0][0] == '*' || grid[n-1][n-1] == '*') return 0;
11     // Space: O(N^2)
12     vector<vector<int>> dp(n, vector<int>(n, 0));
13     dp[0][0] = 1;
14     // Time: O(N^2)
15     for (int row = 0; row < n; ++row) {
16         for (int col = 0; col < n; ++col) {
17             if (grid[row][col] == '*') continue;
18             if (row > 0) dp[row][col] = (dp[row][col] + dp[row - 1][col]) % MOD;
19
20             if (col > 0) dp[row][col] = (dp[row][col] + dp[row][col - 1]) % MOD;
21         }
22     }
23     return dp[n-1][n-1];
24 }

```

```

19         if (col > 0) dp[row][col] = (dp[row][col] + dp[row][col - 1]) % MOD;
20     }
21 }
22 return dp[n-1][n-1];
23 }
24
25 int32_t main() {
26     int n;
27     cin >> n;
28     vector<string> grid(n); // Space: O(N^2)
29     for (int i = 0; i < n; ++i) cin >> grid[i];
30     cout << countGridPaths(n, grid) << "\n";
31     return 0;
32 }

```

### 13.3. Rectangle Cutting [CSES]

**Concept:** Split a WxH rectangle into squares. Try all horizontal and vertical cuts.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int getMinCuts(int width, int height) {
8     // Space: O(W * H)
9     vector<vector<int>> dp(width + 1, vector<int>(height + 1, 1e9));
10    // Time: O(W * H * (W + H))
11    for (int w = 1; w <= width; ++w) {
12        for (int h = 1; h <= height; ++h) {
13            if (w == h) {
14                dp[w][h] = 0;
15                // Already a square
16                continue;
17            }
18            // Vertical cuts
19            for (int cut = 1; cut < w; ++cut) {
20                dp[w][h] = min(dp[w][h], dp[cut][h] + dp[w - cut][h] + 1);
21            }
22            // Horizontal cuts
23            for (int cut = 1; cut < h; ++cut) {
24                dp[w][h] = min(dp[w][h], dp[w][cut] + dp[w][h - cut] + 1);
25            }
26        }
27    }
28    return dp[width][height];
29 }

```

### 13.4. Minimum Sum in a Triangle

**Concept:** Move from top to bottom. Optimized by going bottom-up, reducing space to 1D.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int getMinTrianglePath(const vector<vector<int>>& triangle) {
8     int rows = triangle.size();
9     if (rows == 0) return 0;
10    // Space: O(N) where N is the number of rows
11    // Initialize the dp array with the bottom row of the triangle
12    vector<int> dp = triangle[rows - 1];
13    // Time: O(N^2)
14    // Work upwards from the second-to-last row
15    for (int r = rows - 2; r >= 0; --r) {
16        for (int c = 0; c <= r; ++c) {
17            // Path cost = Current cell + min of the two cells directly below
18            // it
19            dp[c] = triangle[r][c] + min(dp[c], dp[c + 1]);
20        }
21    }
22    return dp[0];
23    // The top element now holds the minimum path sum
24 }

```

### 13.5. Maximal Square (All 1s)

**Concept:** Find the largest square sub-matrix containing only 1s.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int getMaximumSquareArea(const vector<vector<int>>& matrix) {
8     if (matrix.empty()) return 0;
9     int rows = matrix.size();
10    int cols = matrix[0].size();
11    int max_side = 0;
12    // Space: O(R * C)
13    vector<vector<int>> dp(rows + 1, vector<int>(cols + 1, 0));
14    // Time: O(R * C)
15    for (int i = 1; i <= rows; ++i) {
16        for (int j = 1; j <= cols; ++j) {
17            // Assuming the matrix contains integers 0 or 1
18            if (matrix[i - 1][j - 1] == 1) {
19                // The size of the square ending at this cell is constrained by
20                // the smallest square above it, left of it, and top-left of it
21                dp[i][j] = min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]})
22                    + 1;
23                max_side = max(max_side, dp[i][j]);
24            }
25        }
26    }
27    return max_side * max_side;
28 }

```

```

24     }
25 }
26 return max_side * max_side;
27 }

```

## 14. Subsequences and Strings

### 14.1. Increasing Subsequence [CSES]

**Concept:** Classic Longest Increasing Subsequence (LIS). Solved in  $O(N \log N)$  using `std::lower_bound` rather than  $O(N^2)$  DP array.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int getLISLength(int n, const vector<int>& arr) {
8     // Space: O(N)
9     vector<int> lis_tails;
10    // Time: O(N log N)
11    for (int i = 0; i < n; ++i) {
12        auto it = lower_bound(lis_tails.begin(), lis_tails.end(), arr[i]);
13        if (it == lis_tails.end()) {
14            lis_tails.push_back(arr[i]);
15            // Extend the subsequence
16        } else {
17            *it = arr[i];
18            // Replace to maintain smallest possible tail
19        }
20    }
21    return lis_tails.size();
22 }

```

### 14.2. Problem E: Increasing Subsequence [Kattis]

**Concept:** The problem specifically asks for the *lexicographically smallest* sequence in the case of a tie. Because  $N \leq 200$ , we can use an  $O(N^2)$  approach and trace backward carefully.

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 void printLexicographicallySmallestLIS(int n, const vector<int>& arr) {
8     // dp[i] = length of LIS starting at index i. Space: O(N)
9     vector<int> dp(n, 1);
10    vector<int> next_index(n, -1);
11
12    int max_length = 0;
13
14    for (int i = n - 1; i >= 0; --i) {
15        for (int j = i + 1; j < n; ++j) {
16            if (arr[j] > arr[i] && dp[j] + 1 > dp[i]) {
17                dp[i] = dp[j] + 1;
18                next_index[i] = j;
19            }
20        }
21    }
22
23    int start = 0;
24    for (int i = 0; i < n; ++i) {
25        if (dp[i] == max_length) {
26            start = i;
27            break;
28        }
29    }
29
30    while (start < n) {
31        cout << arr[start] << " ";
32        start = next_index[start];
33    }
34    cout << endl;
35 }

```

```

14 // Time: O(N^2). We iterate backwards to easily find the first
    // lexicographical match.
15 for (int i = n - 1; i >= 0; --i) {
16     int best_next = -1;
17     for (int j = i + 1; j < n; ++j) {
18         if (arr[j] > arr[i]) {
19             // If it creates a longer sequence, OR it ties but the element
            // is smaller
20             if (dp[j] + 1 > dp[i] || (dp[j] + 1 == dp[i] && arr[j] < arr[
                best_next])) {
21                 dp[i] = dp[j] + 1;
22                 best_next = j;
23             }
24         }
25     }
26     next_index[i] = best_next;
27     max_length = max(max_length, dp[i]);
28 }
29
30 // Find the best starting element
31 int current_index = -1;
32 for (int i = 0; i < n; ++i) {
33     if (dp[i] == max_length) {
34         if (current_index == -1 || arr[i] < arr[current_index]) {
35             current_index = i;
36         }
37     }
38 }
39
40 cout << max_length << " ";
41 while (current_index != -1) {
42     cout << arr[current_index] << " ";
43     current_index = next_index[current_index];
44 }
45 cout << "\n";
46 }

```

### 14.3. Longest Common Subsequence (LCS)

**Concept:** Find the longest sequence of characters that appear in both strings in the same order.

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5
6 using namespace std;
7
8 int getLCS(const string& text1, const string& text2) {
9     int len1 = text1.length();
10    int len2 = text2.length();
11
12    // Space: O(N * M)
13    vector<vector<int>> dp(len1 + 1, vector<int>(len2 + 1, 0));

```

```

14 // Time: O(N * M)
15 for (int i = 1; i <= len1; ++i) {
16     for (int j = 1; j <= len2; ++j) {
17         if (text1[i - 1] == text2[j - 1]) {
18             dp[i][j] = dp[i - 1][j - 1] + 1;
19             // Characters match
20         } else {
21             dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
22             // Skip one character
23         }
24     }
25 }
26 return dp[len1][len2];
27 }

```

### 14.4. Word Break Problem

**Concept:** Can a string be fully segmented into dictionary words?

```

1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <unordered_set>
5
6 using namespace std;
7
8 bool canBreakWord(const string& target, const vector<string>& word_dict) {
9     unordered_set<string> dict(word_dict.begin(), word_dict.end());
10    int length = target.length();
11    // Space: O(N) - dp[i] is true if the prefix of length 'i' can be segmented
12    vector<bool> dp(length + 1, false);
13    dp[0] = true; // Empty string is always valid
14
15    // Time: O(N^2) (Assuming substring extraction is O(1) conceptually, O(N^3)
    // in strict C++)
16    for (int i = 1; i <= length; ++i) {
17        for (int j = 0; j < i; ++j) {
18            // If the prefix up to 'j' is valid, and the rest [j..i] is in the
            // dictionary
19            if (dp[j] && dict.count(target.substr(j, i - j))) {
20                dp[i] = true;
21                break; // No need to check other splits for this 'i'
22            }
23        }
24    }
25    return dp[length];
26 }

```

## 15. Graphs and Short Path [DP]

### 15.1. Bellman-Ford Algorithm

**Concept:** A graph DP algorithm to find the shortest path from a single source to all vertices. Unlike Dijkstra, it correctly handles negative edge weights and detects negative



weight cycles.

**State:** After the  $k$ -th iteration, the distance array contains the shortest path using at most  $k$  edges.

```

1  #include <iostream>
2  #include <vector>
3  #include <climits>
4
5  using namespace std;
6
7  struct Edge {
8      int source;
9      int destination;
10     int weight;
11 };
12
13 // Returns an empty vector if a negative cycle is detected
14 vector<long long> bellmanFord(int num_vertices, int source_vertex, const vector
    <Edge>& edges) {
15     // Space Complexity: O(V)
16     // Use long long to prevent overflow when adding to INT_MAX
17     vector<long long> distances(num_vertices, INT_MAX);
18     distances[source_vertex] = 0;
19
20     // Time Complexity: O(V * E) - Relax all edges V - 1 times
21     for (int i = 0; i < num_vertices - 1; ++i) {
22         bool any_updated = false;
23         for (const Edge& edge : edges) {
24             if (distances[edge.source] != INT_MAX &&
25                 distances[edge.source] + edge.weight < distances[edge.
26                     destination]) {
27
28                 distances[edge.destination] = distances[edge.source] + edge.
29                     weight;
30                 any_updated = true;
31             }
32         }
33         // Early stopping optimization: if no distances changed, we are done
34         if (!any_updated) break;
35     }
36
37     // Time Complexity: O(E) to check for negative cycles
38     // If we can STILL relax an edge after V-1 passes, a negative cycle exists.
39     for (const Edge& edge : edges) {
40         if (distances[edge.source] != INT_MAX &&
41             distances[edge.source] + edge.weight < distances[edge.destination])
42         {
43             // Negative cycle detected!
44             return {};
45         }
46     }
47
48     return distances;
49 }

```

## 15.2. Problem F & G: Bellman-Ford with Negative Cycles [Kattis]

**Concept:** *Haunted Graveyard* and *Single Source Shortest Path* are both fundamentally testing your ability to deploy Bellman-Ford and correctly identify nodes trapped in negative cycles. The template below handles the complete requirement: finding shortest paths, detecting negative cycles, and safely marking *every node reachable* from that negative cycle as  $-\infty$ .

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  #define int long long
6
7  struct Edge {
8      int u, v, weight;
9  };
10
11 void solveBellmanFordNegativeCycles(int num_vertices, int num_edges, int source
    , int num_queries) {
12     vector<Edge> edges(num_edges);
13     for (int i = 0; i < num_edges; ++i) {
14         cin >> edges[i].u >> edges[i].v >> edges[i].weight;
15     }
16
17     // Space: O(V)
18     vector<int> distances(num_vertices, 1e15);
19     // 1e15 represents Infinity
20     distances[source] = 0;
21
22     // Step 1: Relax edges V-1 times. Time: O(V * E)
23     for (int i = 0; i < num_vertices - 1; ++i) {
24         for (const Edge& e : edges) {
25             if (distances[e.u] != 1e15 && distances[e.u] + e.weight < distances
26                 [e.v]) {
27                 distances[e.v] = distances[e.u] + e.weight;
28             }
29         }
30     }
31
32     // Step 2: Propagate Negative Infinity. Run V-1 more times.
33     // If a node can STILL be relaxed, it is part of, or reachable from, a
34     // negative weight cycle.
35     // Time: O(V * E)
36     for (int i = 0; i < num_vertices - 1; ++i) {
37         for (const Edge& e : edges) {
38             if (distances[e.u] != 1e15 && distances[e.u] + e.weight < distances
39                 [e.v]) {
40                 distances[e.v] = -1e15; // Mark as -Infinity
41             }
42         }
43     }
44
45     // Answer queries
46     for (int q = 0; q < num_queries; ++q) {

```

```

44     int destination;
45     cin >> destination;
46
47     if (distances[destination] == 1e15) {
48         cout << "Impossible\n";
49     } else if (distances[destination] == -1e15) {
50         cout << "-Infinity\n";
51     } else {
52         cout << distances[destination] << "\n";
53     }
54 }
55 }

```

### 15.3. Floyd-Warshall (All-Pairs Shortest Path)

**Concept:** A  $O(V^3)$  DP that finds the shortest distance between *every* pair of vertices in a graph.

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  void runFloydWarshall(vector<vector<int>>& distance_matrix) {
8      int num_vertices = distance_matrix.size();
9
10     // Time:  $O(V^3)$ . The core DP logic is incredibly simple but computationally
11     // heavy.
12     // 'k' represents the intermediate vertex we are trying to route through.
13     for (int k = 0; k < num_vertices; ++k) {
14         for (int i = 0; i < num_vertices; ++i) {
15             for (int j = 0; j < num_vertices; ++j) {
16                 // If vertex 'k' is reachable from 'i' and 'j' is reachable
17                 // from 'k'
18                 if (distance_matrix[i][k] != 1e9 && distance_matrix[k][j] != 1
19                     e9) {
20                     // Update shortest path from 'i' to 'j'
21                     distance_matrix[i][j] = min(
22                         distance_matrix[i][j],
23                         distance_matrix[i][k] + distance_matrix[k][j]
24                     );
25                 }
26             }
27         }
28     }
29 }

```

## 16. Mathematics and State Machines

### 16.1. Array Description [CSES]

**State:**  $dp[i][v]$  = ways to fill prefix  $i$  such that the last element is  $v$ .

```

1  #include <iostream>

```

```

2  #include <vector>
3
4  using namespace std;
5  #define int long long
6  const int MOD = 1e9 + 7;
7
8  int countValidArrays(int n, int max_val, const vector<int>& arr) {
9      // Space:  $O(N * M)$ 
10     vector<vector<int>> dp(n, vector<int>(max_val + 2, 0));
11     // Base case for first element
12     if (arr[0] != 0) {
13         dp[0][arr[0]] = 1;
14     } else {
15         for (int v = 1; v <= max_val; ++v) dp[0][v] = 1;
16     }
17
18     // Time:  $O(N * M)$ 
19     for (int i = 1; i < n; ++i) {
20         if (arr[i] != 0) {
21             int v = arr[i];
22             dp[i][v] = (dp[i-1][v-1] + dp[i-1][v] + dp[i-1][v+1]) % MOD;
23         } else {
24             for (int v = 1; v <= max_val; ++v) {
25                 dp[i][v] = (dp[i-1][v-1] + dp[i-1][v] + dp[i-1][v+1]) % MOD;
26             }
27         }
28     }
29
30     int total_ways = 0;
31     for (int v = 1; v <= max_val; ++v) {
32         total_ways = (total_ways + dp[n-1][v]) % MOD;
33     }
34     return total_ways;
35 }

```

### 16.2. Removal Game [CSES]

**Concept:** Minimax DP. Players take from ends. We track the *difference* (Player 1 Score - Player 2 Score).

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6  #define int long long
7
8  int getPlayerOneMaxScore(int n, const vector<int>& arr) {
9      int total_sum = 0;
10     for (int x : arr) total_sum += x;
11
12     // Space:  $O(N^2)$ 
13     vector<vector<int>> dp(n, vector<int>(n, 0));
14     for (int i = 0; i < n; ++i) dp[i][i] = arr[i];
15     // Time:  $O(N^2)$ 

```

```

16     for (int len = 2; len <= n; ++len) {
17         for (int left = 0; left <= n - len; ++left) {
18             int right = left + len - 1;
19             // Maximize my score minus the opponent's best response
20             dp[left][right] = max(arr[left] - dp[left + 1][right],
21                                 arr[right] - dp[left][right - 1]);
22         }
23     }
24
25     // Player 1 score is (Total Sum + Difference) / 2
26     return (total_sum + dp[0][n - 1]) / 2;
27 }

```

### 16.3. Catalan Numbers (BSTs, Parentheses, Triangulations)

**Concept:** The  $n$ -th Catalan number counts many structural combinations, such as the number of unique BSTs you can make with  $n$  nodes.

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  #define int long long
6
7  int getCatalanNumber(int n) {
8      if (n <= 1) return 1;
9      // Space: O(N)
10     vector<int> dp(n + 1, 0);
11     dp[0] = 1;
12     dp[1] = 1;
13     // Time: O(N^2)
14     for (int i = 2; i <= n; ++i) {
15         for (int j = 0; j < i; ++j) {
16             dp[i] += dp[j] * dp[i - j - 1];
17         }
18     }
19     return dp[n];
20 }

```

### 16.4. Count Derangements

**Concept:** Permutations where no element remains in its original position.

**State Transition:** Item  $i$  can swap with any of the  $(i - 1)$  previous items.

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  #define int long long
6  const int MOD = 1e9 + 7;
7
8  int countDerangements(int n) {
9      if (n == 1) return 0;
10     if (n == 2) return 1;
11     // Space: O(1) - Only need the last two states

```

```

12     int prev2 = 0;
13     int prev1 = 1;
14
15     // Time: O(N)
16     for (int i = 3; i <= n; ++i) {
17         int current = (i - 1) * (prev1 + prev2) % MOD;
18         prev2 = prev1;
19         prev1 = current;
20     }
21     return prev1;
22 }

```

### 16.5. Binomial Coefficient (Pascal's Triangle 1D)

**Concept:** Compute  $\binom{n}{k}$  efficiently using  $O(K)$  space instead of an  $O(N \times K)$  matrix.

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5  #define int long long
6
7  int getBinomialCoefficient(int n, int k) {
8      if (k > n) return 0;
9      if (k == 0 || k == n) return 1;
10     // Optimize: nCr(n, k) == nCr(n, n - k)
11     if (k > n - k) k = n - k;
12     // Space: O(K)
13     vector<int> dp(k + 1, 0);
14     dp[0] = 1;
15     // Time: O(N * K)
16     for (int i = 1; i <= n; ++i) {
17         // Traverse backwards to avoid overwriting values needed for current
18         // row
19         for (int j = min(i, k); j > 0; --j) {
20             dp[j] = dp[j] + dp[j - 1];
21         }
22     }
23     return dp[k];
24 }

```

### 16.6. Decode Ways

**Concept:** Count ways to decode a string of digits where A=1, B=2 ... Z=26.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4
5  using namespace std;
6
7  int numDecodings(const string& digits) {
8      if (digits.empty() || digits[0] == '0') return 0;
9      int length = digits.length();
10     // Space: O(N)

```

```

11     vector<int> dp(length + 1, 0);
12
13     dp[0] = 1; // Empty string is 1 valid sequence
14     dp[1] = 1;
15     // Time: O(N)
16     for (int i = 2; i <= length; ++i) {
17         // Check single digit (1-9)
18         int one_digit = stoi(digits.substr(i - 1, 1));
19         if (one_digit >= 1 && one_digit <= 9) {
20             dp[i] += dp[i - 1];
21         }
22
23         // Check two digits (10-26)
24         int two_digits = stoi(digits.substr(i - 2, 2));
25         if (two_digits >= 10 && two_digits <= 26) {
26             dp[i] += dp[i - 2];
27         }
28     }
29     return dp[length];
30 }

```

## 17. Advanced Topics (Bitmask and Intervals)

### 17.1. Projects (Weighted Interval Scheduling) [CSES]

**Concept:** Sort jobs by end time. Use binary search (`lower_bound`) to find the latest non-overlapping project.

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6  #define int long long
7
8  struct Project {
9      int start, end, reward;
10 };
11
12 bool compareProjects(const Project& a, const Project& b) {
13     return a.end < b.end;
14 }
15
16 int getMaxReward(int n, vector<Project>& projects) {
17     // Time: O(N log N)
18     sort(projects.begin(), projects.end(), compareProjects);
19     // Space: O(N)
20     vector<int> dp(n, 0);
21     vector<int> end_times(n);
22     for (int i = 0; i < n; ++i) end_times[i] = projects[i].end;
23
24     dp[0] = projects[0].reward;
25     // Time: O(N log N)
26     for (int i = 1; i < n; ++i) {
27         int include_reward = projects[i].reward;

```

```

28         // Find the last project that ended before the current one started
29         auto it = lower_bound(end_times.begin(), end_times.end(), projects[i].
30             start);
31         if (it != end_times.begin()) {
32             --it;
33             int last_valid_idx = distance(end_times.begin(), it);
34             include_reward += dp[last_valid_idx];
35         }
36         dp[i] = max(dp[i - 1], include_reward);
37     }
38     return dp[n - 1];
39 }

```

### 17.2. Elevator Rides (Bitmask DP) [CSES]

**Concept:** Minimize elevator trips for  $N$  people ( $N \leq 20$ ). We use a bitmask where the  $i$ -th bit represents whether person  $i$  is in the elevator.

**State:**  $dp[mask] = \{\text{minimum rides, weight of current ride}\}$

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6  #define int long long
7
8  struct State {
9      int rides;
10     int current_weight;
11 };
12
13 int getMinElevatorRides(int n, int max_weight, const vector<int>& weights) {
14     int max_mask = (1 << n);
15     // Space: O(2^N)
16     vector<State> dp(max_mask, {25, 0});
17     // 25 is safe infinity for N=20
18
19     dp[0] = {1, 0};
20     // Time: O(N * 2^N)
21     for (int mask = 1; mask < max_mask; ++mask) {
22         for (int i = 0; i < n; ++i) {
23             // If the i-th person is in this subset (mask)
24             if (mask & (1 << i)) {
25                 int previous_mask = mask ^ (1 << i);
26                 State prev_state = dp[previous_mask];
27
28                 State new_state;
29                 if (prev_state.current_weight + weights[i] <= max_weight) {
30                     // Add person to the current ride
31                     new_state = {prev_state.rides, prev_state.current_weight +
32                         weights[i]};
33                 } else {
34                     // Requires a new ride
35                     new_state = {prev_state.rides + 1, weights[i]};

```

```
35         }
36
37         // Keep the state with fewer rides, or less weight on the
           current ride
38         if (new_state.rides < dp[mask].rides ||
39             (new_state.rides == dp[mask].rides && new_state.
               current_weight < dp[mask].current_weight)) {
40             dp[mask] = new_state;
41         }
42     }
43 }
44 }
45 return dp[max_mask - 1].rides;
46 }
```

---